

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS



Construcción de Software I (Fundamentos, Prácticas)

Proyecto 4: Sistema de Gestión Académica

DOCENTE: Mag. Juan Luis Herencia Guerra

GRUPO: 4

- | | |
|-----------------------------------|-----------|
| • Chavez Miranda Leonardo Gabriel | 20240110J |
| • Ramos Jacay Dair David | 20232156D |
| • Eustaquio Avila Luis Gabriel | 20240206G |
| • Paucar Ventura Luis Antonio | 20240124K |
| • Chupa Ballesteros Gabriel | 20240160G |
-

2026-I

1. Introducción

1.1 Propósito

Este documento corresponde al Plan de Implementación del Sistema de Gestión Académica (SGA), su propósito es establecer la hoja de ruta técnica y organizativa que el equipo seguirá para construir el sistema entre la semana 9 y la semana 13 del ciclo 2026-I, manteniendo coherencia con la arquitectura, los requerimientos funcionales y el modelo de datos definidos en el Entregable 1.

El plan abarca las siguientes áreas: la selección y justificación de las tecnologías utilizadas, la estructura del proyecto, la estrategia de trabajo con Git, la distribución de módulos entre los integrantes del equipo, el cronograma de desarrollo, el orden en que se construirá cada módulo según sus dependencias, las convenciones de codificación adoptadas y los riesgos técnicos identificados junto a sus medidas de mitigación.

1.2 Alcance

El plan abarca la implementación completa de los diez componentes funcionales definidos en el documento: Autenticación, Gestión Institucional (Tenant), Períodos Académicos y Políticas, Oferta Académica, Matrícula, Calificaciones, Cierre Académico, Cuentas de Seguimiento, Historial Académico y Auditoría.

La implementación cubre las capas de Aplicación (FastAPI), Persistencia (Supabase/PostgreSQL) y Presentación (React). El despliegue en producción se documenta en el documento complementario “Guía de Despliegue y Operaciones”.

2. Stack Tecnológico

2.1 Selección y justificación

Capa	Tecnología	Versión	Justificación
Backend — Aplicación y Dominio	FastAPI	Python 3.12	Framework ASGI con soporte nativo async, tipado estático vía Pydantic, generación automática de OpenAPI. Adecuado para implementar la capa de dominio con validaciones explícitas por modelo. Soporta middleware para control de Tenant y JWT en cada request.
Presentación	React	18.x / Vite 5.x	SPA con enrutamiento por rol. La capa de presentación solo consume la API REST de FastAPI; no realiza llamadas directas a Supabase, preservando el aislamiento de la capa de dominio definido en RNFMAN-01.
Persistencia	Supabase (PostgreSQL 15)	Cloud	Plataforma BaaS sobre PostgreSQL que provee instancia gestionada, backups automáticos y conexión via pool. Se accede exclusivamente desde FastAPI mediante el driver asyncpg/psycopg3. El cliente Supabase JS no se utiliza en el frontend para preservar la arquitectura en capas.
Despliegue backend	Render	Web Service	Plataforma PaaS con soporte nativo para aplicaciones Python/ASGI. Despliegue continuo desde rama main de GitHub. Vercel no se usa para el backend por ser serverless y no soportar servidores ASGI persistentes requeridos por FastAPI.
Despliegue frontend	Vercel	SPA hosting	Plataforma optimizada para SPAs React. Despliegue continuo desde rama main. Variables de entorno configuradas para apuntar al backend en Render.
Control de versiones	Git + GitHub	Git	Repositorio único (monorepo) con directorios /backend y /frontend. Gestión de ramas por módulo y revisión por pull requests.
Entorno desarrollo	Docker + Docker Compose	Docker	Contenedor para el backend FastAPI en desarrollo local, garantizando entorno reproducible entre los cinco integrantes. No se usa en producción.

2.2 Decisión arquitectónica crítica: acceso a base de datos

El frontend React no utiliza el Supabase JavaScript client para realizar operaciones directas sobre la base de datos. Esta decisión es un invariante arquitectónico derivado del principio RNFMAN-01 y del principio de Aislamiento Institucional como Invariante Estructural definido en SRS-ARQ-SGA-2026.

Si el frontend accediera directamente a Supabase, las validaciones de Tenant, las reglas de negocio (verificación de prerequisites, límites de crédito, estados de período) y el registro de auditoría podrían ser omitidas. Toda operación de lectura y escritura pasa obligatoriamente por la capa de dominio en FastAPI, que valida el contexto institucional antes de ejecutar cualquier operación sobre la persistencia.

3. Estructura del Proyecto

3.1 Organización del repositorio

El proyecto se organiza como monorepo con dos subdirectorios principales, reflejando la separación entre capas de presentación y de aplicación/dominio definida en la arquitectura.

Ruta	Responsabilidad
sga/	Raíz del monorepo
backend/	Capa de Aplicación + Dominio (FastAPI)
app/	Paquete principal de la aplicación
api/	Routers REST por módulo funcional (v1/auth, v1/tenants, v1/periodos, etc.)
services/	Capa de aplicación: coordinación de flujos de negocio por módulo
domain/	Reglas de negocio puras: validaciones, cálculos de promedio, evaluación de prerrequisitos
repositories/	Acceso a datos: consultas SQL hacia Supabase/PostgreSQL
models/	Modelos Pydantic: schemas de entrada/salida por módulo
core/	Configuración, middleware de Tenant, JWT, manejo de errores global
tests/	Pruebas unitarias e integrales por módulo (pytest)
docker-compose.yml	Orquestación del entorno de desarrollo local
Dockerfile	Imagen del backend para desarrollo local
requirements.txt	Dependencias Python fijadas a versión exacta
frontend/	Capa de Presentación (React + Vite)
src/pages/	Páginas por rol: /admin, /docente, /alumno
src/components/	Componentes reutilizables de UI
src/services/	Clientes HTTP hacia la API de FastAPI (axios)
src/context/	Context de autenticación: token JWT, rol, tenant

4. Estrategia de Control de Versiones

4.1 Modelo de ramas

Se adopta un modelo de ramas basado en Feature Branching con una rama principal de integración.

Rama	Propósito
main	Rama protegida. Solo recibe merges por Pull Request aprobado. Refleja el estado estable del sistema. Render y Vercel despliegan automáticamente desde esta rama.
develop	Rama de integración continua. Todos los módulos se integran aquí antes de pasar a main.
feature/[módulo]-[descripción]	Rama por funcionalidad. Ejemplos: feature/auth-jwt, feature/matricula-validaciones, feature/calificaciones-registro. Se crea desde develop y se fusiona a develop via PR.
fix/[descripción]	Correcciones de errores identificados durante integración o pruebas.

4.2 Convenciones de commits

Se adopta Conventional Commits para mantener trazabilidad entre commits y módulos del sistema, ejemplos:

- feat(matricula): implementar validación de prerequisites RF-MAT-02
- feat(auth): implementar generación de JWT con contexto de Tenant
- fix(calificaciones): corregir validación de rango en escala de evaluación
- test(periodos): agregar pruebas de transición de estado CONFIGURACION→MATRICULA
- docs(api): actualizar descripción de endpoint POST /matriculas

5. Asignación de Módulos por Integrante

La asignación respeta las dependencias funcionales entre módulos. Cada integrante es responsable de la implementación completa de su módulo: API (routers), servicio (lógica de negocio), repositorio (acceso a datos), modelos Pydantic y pruebas unitarias.

Integrante	Módulos asignados	Código
Chavez Miranda Leonardo Gabriel	Autenticación y Gestión de Sesión Gestión Institucional (Tenant) Gestión de Usuarios	20240110J
Ramos Jacay Dair David	Gestión de Períodos Académicos y Políticas Oferta Académica (Cursos y Secciones)	20232156D
Eustaquio Avila Luis Gabriel	Gestión de Matrícula Cuentas de Seguimiento (integración)	20240206G
Paucar Ventura Luis Antonio	Gestión de Calificaciones Cierre Académico Historial Académico	20240124K
Chupa Ballesteros Gabriel	Capa de Presentación React (frontend completo) Auditoría (integración transversal) Reportes	20240160G

Nota: Los módulos de Cuentas de Seguimiento y Auditoría son transversales. Su implementación de persistencia la lidera el responsable indicado; los demás módulos invocan sus interfaces definidas.

6. Orden de Implementación por Dependencias (Estimado)

El orden de implementación se deriva de las dependencias funcionales estrictas definidas en la arquitectura. Un módulo no puede implementarse antes de que sus dependencias estén disponibles como interfaces estables.

Fase	Módulos	Semana	Dependencias previas requeridas
1	Auth + Tenant + Usuarios Estructura base del proyecto Configuración BD (Supabase)	Semana 9	Ninguna. Base de todo el sistema.
2	Períodos Académicos y Políticas Catálogos Institucionales	Semana 9–10	Tenant activo. Contexto institucional validado en cada operación.
3	Oferta Académica: Cursos, Planes, Secciones, Componentes de Evaluación	Semana 10	Período activo en estado CONFIGURACION o posterior. Catálogos de evaluación disponibles.
4	Matrícula Cuentas de Seguimiento	Semana 10–11	Período en estado MATRICULA. Secciones con cupos configurados. Historial académico consultable.
5	Calificaciones Auditoría (transversal)	Semana 11	Inscripciones activas. Período en estado REGISTRO_NOTAS. Componentes de evaluación configurados.
6	Cierre Académico Historial Académico Reportes	Semana 12	Todos los componentes de evaluación en estado PUBLICADO. Inscripciones con calificaciones completas.
7	Integración frontend Pruebas de integración Correcciones	Semana 12–13	Todos los módulos backend funcionales con pruebas unitarias aprobadas.

7. Cronograma de Implementación

Actividad	S9 (21/05)	S10 (28/05)	S11 (04/06)	S12 (11/06)	S13
Setup: repo, Docker, Supabase, CI/CD	●				
Auth + Tenant + Usuarios (backend)	●				
Períodos y Políticas (backend)	●	●			
Oferta Académica (backend)		●			
Matrícula + Cuentas de Seguimiento (backend)		●	●		
Calificaciones + Auditoría (backend)			●		
Cierre Académico + Historial + Reportes (backend)			●	●	
Pruebas unitarias (por módulo, paralelo)	●	●	●	●	●
Frontend React — vistas y servicios		●	●	●	
Integración frontend–backend + pruebas integrales				●	●
E3 Evidencia de pruebas (04/06)			▲		
E4 Código fuente y documentación (18/06)					▲

● = actividad en curso ▲ = hito de entrega

8. Estándares de Codificación

8.1 Backend (Python / FastAPI)

Aspecto	Convención
Estilo	PEP 8. Formateador: Black (line-length 88). Linter: Ruff.
Tipado	Anotaciones de tipo obligatorias en funciones de servicio y repositorio. Modelos de entrada/salida definidos como clases Pydantic BaseModel.
Nomenclatura	snake_case para variables y funciones. PascalCase para clases. Prefijos por capa: get_*/create_*/update_* en repositorios; handle_*/process_* en servicios.
Validaciones	Las reglas de negocio (validación de prerequisites, límites de crédito, estados de período) residen en app/domain/, no en routers ni en repositorios. Los routers solo validan estructura del request via Pydantic.
Errores	HTTPException con códigos semánticos (400 validación, 403 autorización, 404 recurso, 409 conflicto de estado, 422 schema). Los mensajes son en español, sin exponer trazas internas (RNFUSA-02).
Pruebas	pytest. Cobertura mínima del 80% sobre módulos críticos: validaciones de matrícula (RF-MAT-02, 03, 04), transiciones de período (RF-PER-02) y cálculo de cierre (RF-CAL-01, 02, 03). Requerimiento RNFMAN-03.
Multi-Tenant	El id_tenant se extrae del JWT en middleware y se inyecta en todos los repositorios vía dependency injection. Ningún endpoint acepta id_tenant como parámetro de request.

8.2 Frontend (React)

Aspecto	Convención
Estilo	ESLint + Prettier. Componentes funcionales con hooks. Sin clases.
Nomenclatura	PascalCase para componentes. camelCase para hooks y funciones. Archivos de componentes: NombreComponente.jsx.
Gestión de estado	Context API para autenticación y datos de sesión (token, rol, tenant). React Query para caché de datos remotos.
Comunicación API	Todos los llamados HTTP en src/services/. Los componentes no llaman a axios directamente. El token JWT se adjunta en Authorization header via interceptor de axios.
Rutas protegidas	Componente PrivateRoute verifica token y rol antes de renderizar. Redirección a /login si no autenticado. Redirección a /unauthorized si rol insuficiente.

9. Gestión de Riesgos Técnicos

Riesgo	Impacto	Plan de mitigación	Responsable
Acceso directo al Supabase client desde el frontend (bypass de capa de dominio)	Alto	Decisión arquitectónica documentada y revisada en PR. El SDK de Supabase JS no se instala en el frontend.	Chavez Miranda / Chupa Ballesteros
Reglas de negocio implementadas en routers o en BD en lugar de en domain/	Alto	Revisión obligatoria de PRs para módulos críticos. Checklist de revisión incluye verificación de ubicación de lógica.	Todo el equipo
Inconsistencia de entorno de desarrollo entre integrantes	Medio	Docker Compose con imagen fija de Python 3.12. requirements.txt con versiones exactas.	Chavez Miranda
Conflictos de merge en módulos con dependencias cruzadas	Medio	Interfaces de módulo (contratos de función en services/) definidas en semana 9 antes de implementación. Comunicación vía issue en GitHub.	Todo el equipo
Cierre académico asíncrono no implementado a tiempo para E3	Medio	Implementar primero versión síncrona funcional; refactorizar a asíncrona (BackgroundTasks de FastAPI) en semana 12.	Paucar Ventura
Límite de conexiones gratuitas a Supabase en desarrollo simultáneo	Bajo	Un único proyecto Supabase de desarrollo compartido por el equipo. Variables de entorno por integrante para el schema de pruebas.	Ramos Jacay